Cahier des charges - Déploiement de Logistico-train

Logistico-train est un système applicatif de gestion des arrivées, sorties et réparations de rames train dans un centre de maintenance.

Le système est distribué sur plusieurs applications. Actuellement ces applications sont hebergées sur d'anciens serveurs et l'entreprise peine à les maintenir à jour, à assurer la sécurité et à garantir la disponibilité et la montée en charge.

L'entreprise souhaite migrer son déploiement sur une infrastructure Docker, ou chaque composant applicatif du système sera déployé dans un conteneur.

Fonctionnement général de la gestion des rames dans un centre de maintenance

Lorsqu'une rame de train arrive au centre, le conducteur envoie une demande d'entrée au centre. Cette demande contient :

- le numéro série de la rame (ex.: AZ4356763), unique à chaque rame;
- l'identifiant de l'auteur de la demande (le conducteur, ex.: TR007) ;
- le type de la rame (ex.: BB7200);
- la liste de tâche de maintenance à réaliser (ex.: ["Repeindre porte S1", "changer néon défaillant voiture centre", "vérifier système hydrolique porte"]). Cette dernière comporte au moins une tâche.

L'opérateur principal du centre recoit la demande en temps réel et l'affecte à une voie disponible. Si aucune voie n'est disponible sur le moment, la demande est refusée, la rame est mise en attente et sera l'objet d'une nouvelle demande d'entrée a posteriori.

Lorsque l'opérateur a définie une voie pour la rame, la demande est alors complétée avec les informations suivantes :

- numéro de voie (une voie ne peut accueillir qu'une seule rame à la fois)

Lorsque qu'un technicien agit sur une rame du dépôt, celui-ci enregistre la ou les tâches qu'il a réalisées, qui sont alors retirées des tâches à réaliser. Des nouvelles tâches peuvent être ajoutées lors de l'entretien si le technicien en découvre le besoin. Lorsque plus aucune tâche n'est à réaliser, la rame est considérée disponible au retrait et un conducteur peut alors la sortir du dépôt. La voie sur laquelle était la rame est alors de nouveau disponible.

Enfin l'opérateur principal du centre peut administrer les voies de ce centre : en ajouter de nouvelles, en retirer, si celles-ci ne sont pas occupées par une rame, et en marquer certaines comme "interdite", si celles-ci ne sont pas également occupées par une rame (parce que des travaux sont menés dessus par exemple). Tant qu'une voie est "interdite", elle ne pourra pas reçevoir de rame.

L'application utilise un double modèle de données : **un modèle de production** qui représente l'état actuel du centre (voies, rames en demande d'entrée ou en cours d'entretien) et **un modèle d'historique** des actions qui enregistre chaque opération dans l'entrepôt concernant les rames (demande d'entrée d'une

rame, acceptation ou refus d'une demande, tâche(s) réalisée(s) ou ajoutée(s) sur une rame, sortie d'une rame). Chaque action est horodatée et consignée (l'auteur est connue).

Modèle de production

Ce modèle, relationnel, permet de gérer les demandes d'entrée de rame et leurs tâches en court.

Le modèle comporte X tables :

- Rame: modélise les rames en entrance vers le centrance, ou a quai ;
- Tache : modélise les tâches restantes à réaliser sur une rame ;
- Voie : liste des voies de l'entrepot, avec un marqueur d'interdiction d'utilisation de la voie (ex.: travaux, problème d'accès. interdiction n'a rien à voir avec le fait que la voie soit occupée par une rame).

Les tables sont construites ainsi

Voie

- num_voie: INT(11), NOT NULL (PK)
- interdite: TINYINT(1), NOT NULL

Rame

- num_serie: VARCHAR(12), NOT NULL (PK)
- type_rame: VARCHAR(50), NOT NULL
- voie: INT(11), FK (référence Voie.num_voie), UNIQUE
- conducteur_entrant: VARCHAR(50), NOT NULL

Tache

- num_serie_rame: VARCHAR(12), NOT NULL, (PK), FK (référence Rame.num_serie)
- num_tache: INT(11), NOT NULL, (PK)
- tache: TEXT, NOT NULL

Modèle d'historique

Ce modèle non relationnel orienté document est pensé pour stocker chaque action faite sur une rame.

Structure de document :

- **id**: *ObjectId*, not null, définit automatiquement par le SGBD
- **numSerie**: *String*, not null
- **typeRame**: *String*, not null
- **action**: *String*, not null, parmi ["demande", "entree", "rejet", "realTaches", "ajoutTaches", "sortie"]
- **auteur**: *String*, not null, identifiant de l'auteur
- **timestamp**: *Datetime*, not null
- **voie**: *Number*, possiblement absent
    - présent uniquement si action n'est ni "demande" ni "rejet"
- **taches**: *Array<String>*, possiblement absent
    - présent uniquement si action est "demande", "realTache" ou "ajoutTache"

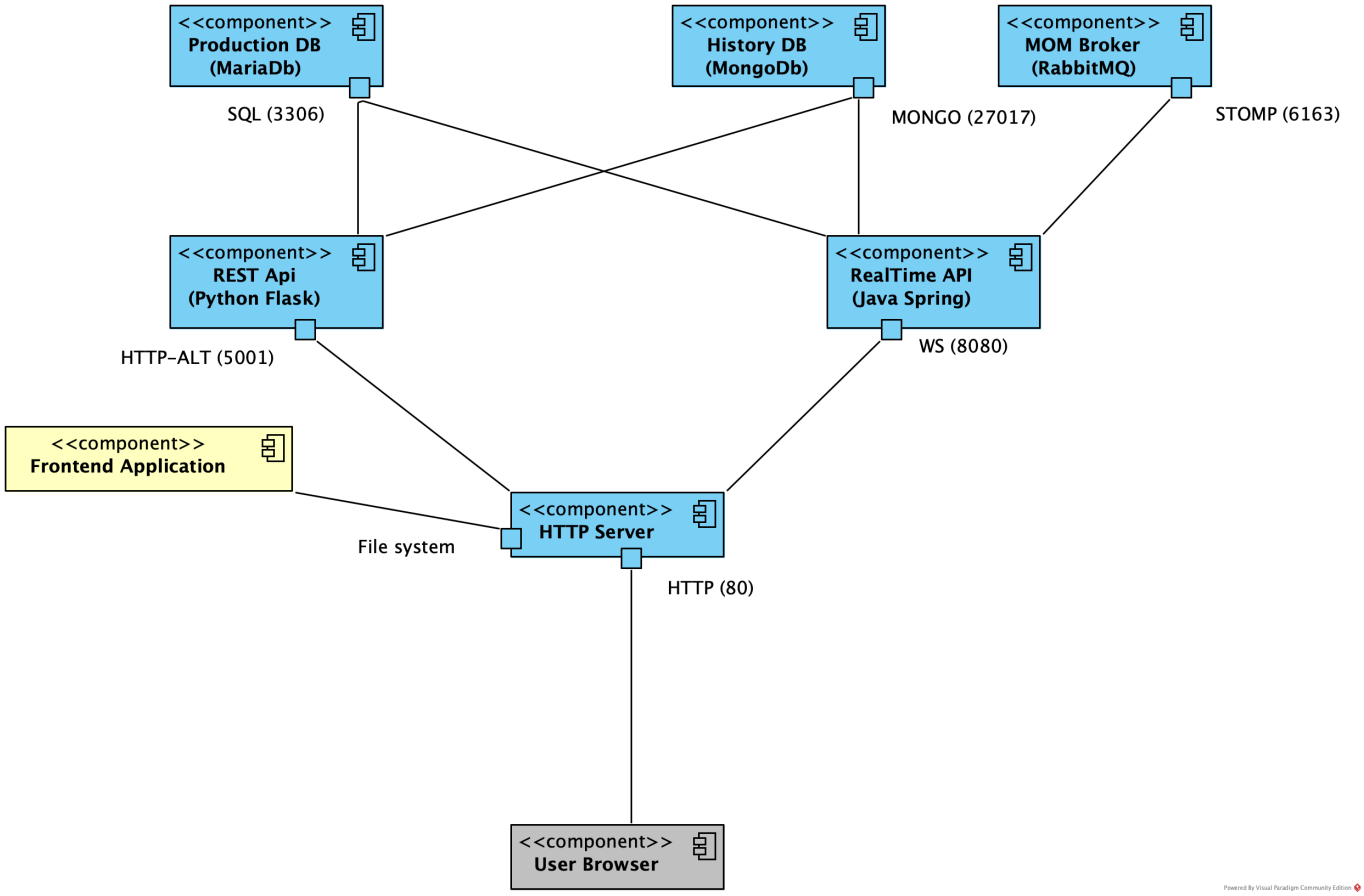
    - si action == "demande" ou "ajoutTache", représente des tâches à réaliser sur la rame
    - si action == "realTaches", représente des tâches réalisées par l'auteur

Manipulation des données

- Lorsqu'un conducteur demande l'entrée au dépot d'une rame :
    - une nouvelle ligne est créé dans la table *Rame* (avec le champs voie à NULL) ;
    - Autant de lignes que de tâches à faire sont créées dans la table *Tache* ;
    - un document d'action "demande" est créée.
- Lorsque l'opérateur de dépôt accepte la rame et l'envoie sur une voie :
    - la ligne *Rame* de la rame est mise à jour avec son champs voie définit ;
    - un document d'action "entree" est créé.
- Lorsque l'opérateur rejête la rame :
    - la ligne de la table *Rame* et ses lignes de *Tache* sont supprimées ;
    - un document d'action "rejet" est créé.
- Lorsqu'un technicien réalise des tâches
    - les tâches réalisées sont retirées de la table *Tache* ;
    - un document d'action "realTaches" est créé avec toutes les tâches qu'il vient de réaliser.
- Lorsqu'un technicien ajoute de noulles tâches a réaliser
    - les tâches sont ajoutée dans la table *Tache* ;
    - un document d'action "ajoutTaches" est créé avec toutes les tâches nouvelles à réaliser.
- Lorsqu'un conducteur sort une rame du dépot (i.e: il n'y a plus de tâche à réaliser) :
    - la Rame est supprimée de la table *Rame* ;
    - un document d'action "sortie" est créé.
- Lorsque l'opérateur administre les voies du dépots
    - des lignes de *Voie* sont ajoutées, modifiées ou retirées selon le traitement.

Architecture de l'application

Cette application multi-tiers est distribuées sur plusieurs composants logiciels



- **Production DB** : base de données de production relationnelle, serveur MariaDb.
- **History DB** : base de données d'historique orientée document, serveur MongoDb.
- **MOM Broker** : Serveur de message temps réel RabbitMQ (assure la haute disponibilité du service temps réél), avec pluging STOMP.
- **REST API** : API REST de gestion du SI, serveur Flask Python. Utilisée pour l'opérateur pour consulter l'état du centre et des rames, gérer les les voies, et par les techniciens pour inscrire la réalisation de tâche ou en ajouter de nouvelles.
- **RealTime API** : Serveur temps réel de WebSocket et point d'entrées REST complémentaire, serveur Spring Java. Utilisée pour demander, accepter/refuser une entrée de rame et sortir une rame. Permet à l'opérateur d'être notifié en temps réel des demandes / entrées / sorties de rame et aux conducteurs ayant demandés une entrée de rame d'être notifié en temps réel de son acceptation ou de son refus.
- **Frontend Application** : ensemble des fichiers sources de l'application web client (html, css, js...).
- **HTTP Server** : Serveur Front HTTP, serveur Nginx. Point d'entrée des clients (navigateurs web). Sert les fichiers statiques issue de la construction de l'application web et fait office de reverse proxy pour les APIs REST et RealTime.

Code-source du système applicatif

Une équipe de développeurs interne a développé 3 applicatifs qui ont été réunis dans un unique dossier qui vous est fourni. Les applicatifs n'ont pas été initialement prévus pour un déploiement sur Docker.

La structure du dépôt est la suivante :

- **app/** : code-source de l'application web client

    - Développée en *React*, avec le builder *Webpack*.
    - Fichiers d'intérêt pour le déploiement :
        - **./package.json** : descripteur de l'application et de ses dépendances npm
        - **./webpack.prod.js** : fichier de configuration de build pour la production
- **RESTApi/** : code-source de l'API REST
    - Développée en *Python*, avec le framework *Flask*
    - Fichiers d'intérêt pour le déploiement :
        - **./README.md** : fichier explicatif
        - **./requirements.txt** : liste des dépendances python
        - **./config.py** : fichier de configuration de l'applicatif
        - **./starter.sh** : script de lancement du serveur
- **RealtimeApi/** : code-source de l'API temps-réel
    - Développée en *Java*, avec le framework *Spring* et le gestionnaire de projet *Maven*
    - Fichiers d'intérêt pour le déploiement :
        - **./README.md** : fichier explicatif
        - **./src/main/resources/application.properties** : fichier de configuration de l'applicatif
        - **./pom.xml** : descripteur de l'application et de ses dépendances Maven
- **vendorConfigurations**: dossier de configurations de composants applicatifs tierce
    - **nginx/nginx.conf** : template configuration du serveur nginx pour le système Logistico-train

Objectifs du déploiement

Vous devez fournir une structure de services docker compose permettant un déploiement de production du système LogisticoTrain.

Structure de base

Il est donc attendu pour la production 6 services **dont les noms vous sont imposés** :

- **sqldatabase** : le serveur de base de données de production. On préférera MariaDb, version 11 minimum.
- **nosqldatabase** : le serveur de base de données d'historique. On préférera Mongodb, version 4.4 minimum, ≥6.0 recommandé.
- **broker** : le serveur de message temps réel. On préférera RabbitMq, version 3.12 minimum.
- **restapi** : le serveur du composant applicatif RESTApi, doit supporter Python 3.11 minimum
- **wsapi** : le serveur du composant applicatif RealtimeApi, doit supporter un JDK Java 21 minimum et maven 3
- **front** : le serveur HTTP Nginx de facade. On préfera Nginx, dans une distribution minimale de sécurité (alpine)

Un 7 service, **webapp**, doit également être prévue pour lancer l'opération de construction de l'application client (**app**) . Toutefois, ce service n'ayant pas pour objectif d'être un serveur, un profile doit être prévu pour ne pas le lancer malencontreusement. Pour le service **webapp** une version de nodeJs ≥ 22 sera nécessaire pour faire le build de l'application.

Soutien au dev et à l'admin

Pour aider les administrateurs système et developpeurs au suivi applicatif, il est également demandé d'adjoindre deux autres services à ne lancer qu'avec un profil distinct (ex.: "dev-tool") :

- un service **phpmyadmin**
- un service **mongo-express**

Ces deux services doit permettre d'accéder rapidement et facilement aux base de données. Il doivent être accessible directement depuis leur port d'accès, mais pas à travers le serveur front nginx. Il ne doivent être accessible que depuis la machine local où sera déployé la pile applicative.

Objectif de résilience du système

- politique de redémarrage automatique sur les bases de données, le broker, les api rest et temps réel, et le serveur front
- vérification de santé sur services critiques : les bases de données, le broker, le serveur front. Chacun devra se faire toutes les 10 secondes, avec un temps d'attente de 10 secondes avant échéc et 10 tentatives max avant de considéré le service comme défaillant
- dependences avancées entre services : les api REST et Realtime ne doivent se lancer que si les bases de données sont lancées et considérées saines. l'api Realtime doit en plus attendre que le broker soit lancé et sain. Le front ne doit se lancer que si les 2 api sont lancées.

Objectif de persistance et performance

- Le composant **RESTApi** n'a pas de roadmap de développement à court terme et sa base de code est considérée stable : il vous est conseillé pour améliorer les performance de faire une image embarquant directement le code source applicatif et si possible ses fichiers python précompilés.
- Le composant RealtimeApi doit a l'inverse pouvoir être redéployé rapidement suite à une modification du code-source : il est conseillé de ne pas faire faire d'image personnalisée, et de monter le code-source par un point de montage.
- Aucun volume de données anonyme ne doit être créé.
- Les bases de données doivent évidemment avoir leurs volumes de données.
- Le service **wsapi** utilise un cache maven pour compiler son code. Il est nécessaire de prévoir un espace de stockage persistant et performant à cet effet.
- Le service **wsapi** compile le code source java dans un espace dédié (dossier target/ du projet, créé à la compilation). Il est nécessaire de s'assurer que cet espace soit également persistant et performant.
- le composant **webapp**, en charge de la construction de l'application cliente, doit pouvoir être exécuté rapidement suite à une modification du code-source : il est conseillé de ne pas faire faire d'image personnalisée, et de monter le code-source par un point de montage.
- L'application cliente est le produit de la compilation effectuée par le service **webapp**. Il est nécessaire de s'assurer que ce build soit persistant.
- Le serveur **front** lira à de nombreuses reprises les fichiers statiques issus du build de l'application client (par **webapp**). Il est nécessaire de s'assurer que cet espace soit performant.
- S'il n'y a pas besoin d'un réseau bridge "par défaut" pour la pile docker compose, réutilisez le réseau bridge par défaut de Docker pour le builder de l'application web cliente (i.e., pas besoin pour docker compose de créer un autre réseau bridge uniquement pour ce service).

Objectif de sécurité

Plusieurs actions de sécurité sont attendues :

- Isolation réseau : il est attendu une segmentation des réseau pour ne laisser un accès aux bases de donnéees qu'aux services qui en ont le besoin, et de même pour le broker.
- Moindre privilige sur les ports ouverts.
- Utilisation de secrets le plus possible pour communiquer les identifiants des bases de données et du broker aux services qui en ont le besoin. On ne veut pas inscrire les identifiants directement dans le fichier de déploiement docker-compose.yml. S'il n'est pas possible d'externaliser les identifiants d'un fichier de configuration d'un service, alors c'est toute la configuration qui doit être considérée comme un secret.
- Lecture seules de volumes et config si il n'y a pas de besoin en écriture pour le service.

Automatisation

- Le modèle de base de données n'est pas créé à la volée. Il est donc nécessaire de préparer la bd en amont. Trouver le moyen de la créer automatiquement, au premier lancement du conteneur **sqldatabase**.
- Le résultat du build de **webapp** doit être automatiquement accessible, en lecture seule, par le service **front**, pour être service au client. les performances en lecture comme en écriture de cet espace de stockage doivent être optimisées.

Livrables attendu :

- **Modèle** de déploiement incluant :
    - plan de services (les différents services, les périmètres de profiles, leur cycle de vie le cas échéant) ;
    - topologie réseau (réseaux, port mapping...) ;
    - plan de données (volumes, bind mount...) ;
    - plan de sécurité (config, secrets).
- **Archive** de déploiement. Dossier content :
    - docker-compose.yml ;
    - structure de dossier et de fichiers de configurations adaptée à la production ;
    - README.md : tout commentaire explicatif, bugs découverts, limite du système à communiquer ;
    - *Vous n'avez pas besoin de remettre tout le code source des composants logiciels dans votre dossier. Mettez des dossiers vides faisant office de placeholders à cet effet.*